JavaScript Essentials: Teaching Web Pages to Think

Introduction to Web Technologies — Week 8

> **The running story.** Picture a web page as a restaurant. HTML is the floor plan and the menu, CSS is the décor, and JavaScript is the **waiter**: it takes your order (*events*), rearranges the tables while you eat (the *DOM*), and walks to the kitchen (the *server*) without freezing the whole dining room while it waits (*asynchronous code*). Keep an eye out for the waiter throughout these notes.

Index

- The Fine Print of JavaScript — every superhero has a weakness; JavaScript has four.
- How a Modern Page Gets Its JavaScript — nine steps from "please send the page" to "wow, it updates itself".
- Where JavaScript Lives in a Page — real estate: embedded, external, or inline (one of these is the good neighbourhood).
- Declaring Variables — `let`, `const`, `var`: labelled boxes with very different trust issues.
- The Data Types of JavaScript — everything is a primitive or an object; there is no third option.
- Output Methods — five ways to make JavaScript talk (and one way to annoy users).
- Operators — arithmetic, comparisons, and why `===` is your best friend.
- Arrays — shelves that grow when you ask nicely.
- Strings — slicing text, and the `"1" + 1` prank.
- Control Flow — forks in the road.
- Loops — doing it again, and again, and again (on purpose).
- Functions — reusable recipes with a name and a return value.
- Scope — who is allowed to see whose variables.
- The Document Object Model — your page as a family tree.
- Finding Elements in the DOM — hide-and-seek with five different search tools.
- Changing the DOM — rearranging the furniture while diners are seated.
- Events — a page that finally listens.
- Practical Scenarios — a click counter and a show/hide panel.
- Client-Side Form Validation — good manners at the door, not a security guard.
- Asynchronous JavaScript — the waiter who never stands idle at the kitchen window.
- Putting It All Together — the whole restaurant, from front door to kitchen, in one paragraph.
- Practice Questions — five brain workouts.

The Fine Print of JavaScript

JavaScript is the language that runs *inside the visitor's browser* (client-side). That is what lets a page react to the user instead of just sitting there. It is a different language from Java despite the similar name. Before learning the syntax, it is worth knowing where JavaScript can let you down.

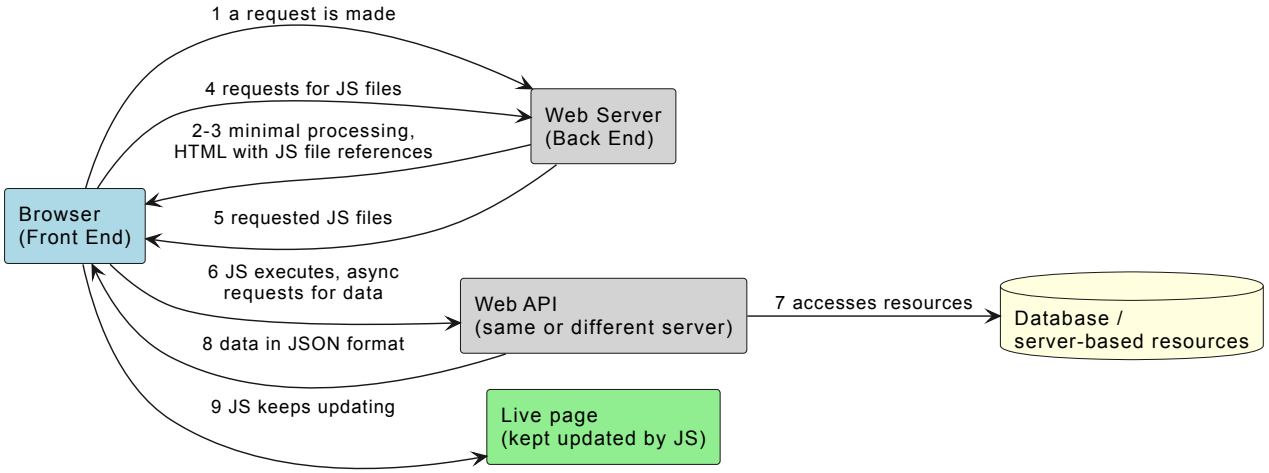
1. **It might be switched off.** There is no guarantee that the client has JavaScript enabled, meaning any required functionality must be implemented redundantly on the server.
2. **It gets hard to manage at scale.** JavaScript-heavy web applications can be complicated to debug and maintain.
3. **It is not fault tolerant.** Browsers are able to handle invalid HTML or CSS. But if your page has invalid JavaScript, it will simply stop execution at the invalid line.
4. **It is a moving target.** While JavaScript is universally supported in all contemporary browsers, the language (and its APIs) is continually being expanded. As such, newer features of the language may not be supported in all browsers.

In restaurant terms: the waiter may call in sick (point 1), so the kitchen must still be able to serve the essentials. HTML and CSS are like a forgiving reader who skips over typos; the JavaScript engine is a strict reader who stops reading aloud at the first mistake (point 3), and everything after that line never runs.

> **Note for Review:** This often appears on exams regarding *client-side vs server-side responsibility*. Remember that anything the page *must* do for correctness (validation, security, access rules) has to be repeated on the server, because the client's JavaScript can be disabled, broken, or bypassed.

How a Modern Page Gets Its JavaScript

In contemporary web development the server does very little work for the first request: it sends a light HTML skeleton that *points to* JavaScript files. The browser then fetches those files, runs them, and the JavaScript asks Web APIs for data and keeps updating the page. The browser side is the **Front End**; the servers and databases are the **Back End**.



The nine steps in words:

1. A request is made by the browser.
2. The server performs minimal processing and returns HTML.
3. The requested HTML arrives with JavaScript file references.
4. The browser makes requests for those JavaScript files.
5. The requested JavaScript files are delivered.
6. JavaScript executes and then makes asynchronous requests for data from web APIs.
7. The Web API accesses server-based resources such as databases.
8. The Web APIs return data in JSON format.
9. JavaScript continually updates the page based on the received data and user actions.

Web APIs could be on the same server or on different servers.

Where JavaScript Lives in a Page

A script can sit in three places relative to the HTML: **embedded** (inside `<script>` tags in the page), **external** (a separate `.js` file linked with `src`), or **inline** (inside an HTML attribute such as `onClick`). Embedded and inline styles are not really recommended; an external file keeps behaviour separate from structure and lets many pages share the same code.

Placement	Where the code is	Verdict
Embedded	Between `<script>` tags in the page	Acceptable for tiny demos
External	In a separate file via `<script src="...">`	**Preferred**: reusable, tidy
Inline	Inside an attribute like `onClick`	Avoid: mixes behaviour into markup

Notice the embedded `alert` sits in the `<head>`, so it fires *before any content* is shown.

```
<html lang="en">
<head>
  <title>JavaScript placement possibilities</title>

  <!-- EMBEDDED JavaScript -->
  <script>
    /* A JavaScript Comment */
    alert("This will appear before any content");
  </script>

  <!-- EXTERNAL JavaScript -->
  <script src="greeting.js"></script>
</head>
<body>
  <h1>Page Title</h1>

  <!-- INLINE JavaScript -->
  <a href="JavaScript:OpenWindow();">for more info</a>
  <input type="button"
         onClick="alert('Are you sure?');" />
</body>
</html>
```

Declaring Variables

A **variable** is a placeholder to which you give a name (an *identifier*); its value is allowed to *vary*. JavaScript offers three keywords for creating one:

- `let` declares a variable you can reassign; `const` declares one you can't (though an object or array it points to can still change internally).
- `var` is the older way of declaring variables — function-scoped rather than block-scoped, which causes real bugs; avoid it in new code.
- JavaScript is dynamically typed: you never declare a type, and a variable can hold a different kind of value later in the script.
- `typeof x` — tells you what type a value currently holds, at runtime.

With `const`, the value is locked: after `const A = 5`, the identifier `A` is always 5. With `let`, the box can be refilled, and because JavaScript is **dynamically typed** the new content need not even be the same kind of thing. A variable can start as an integer, become a string, and then become a decimal number. This is a sharp contrast with C or C++, where the declared type is fixed forever. In JavaScript you "go with the flow": the name simply points to a memory location that accepts any kind of value.

```
const A = 5;
// A will always be 5

let a = 10;      // a number
a = "ten";       // now a string
a = 3.14;        // now a decimal

typeof a;        // "number"
```

A subtle point about `const`: it protects the *binding* (the name cannot be pointed somewhere else), not the *contents* of whatever it points to. An array or object held in a `const` can still be modified internally, but trying to reassign the name itself is an error.

```
const list = [1, 2];
list.push(3);   // allowed: contents change
list = [9];     // error: cannot reassign
```

Scope: How Long Does a Value Live?

Scope defines the range over which a variable (and any change made to it) remains valid. Think of how HTML nests: `body` contains an `h1`, which contains a `p`. Whatever happens inside that innermost element is *local*; step outward and you reach broader and broader regions. Scope works the same way: it defines the lifetime of a variable and how far a change persists. Scope can be driven by a **function** or by simple **braces** (a block), and `var` and `let` differ precisely on this point. The sketch below shows the key idea: a `var` declared inside a block is not confined to it.

```
{
  var a = 10;     // declared inside the block...
}
a = 20;           // ...still usable outside:
                  // var ignores blocks
```

The full story, with four worked cases, is in Scope.

Keyword	Reassignable?	Scope	Use it?
`const`	No	Block	Default choice
`let`	Yes	Block	When the value must change
`var`	Yes	Function	Avoid in new code

> **Note for Review:** Pro-Tip: a classic exam trap is "does `const` make an array immutable?" Remember that `const` freezes the *name-to-value link*, not the *inside* of an array or object.

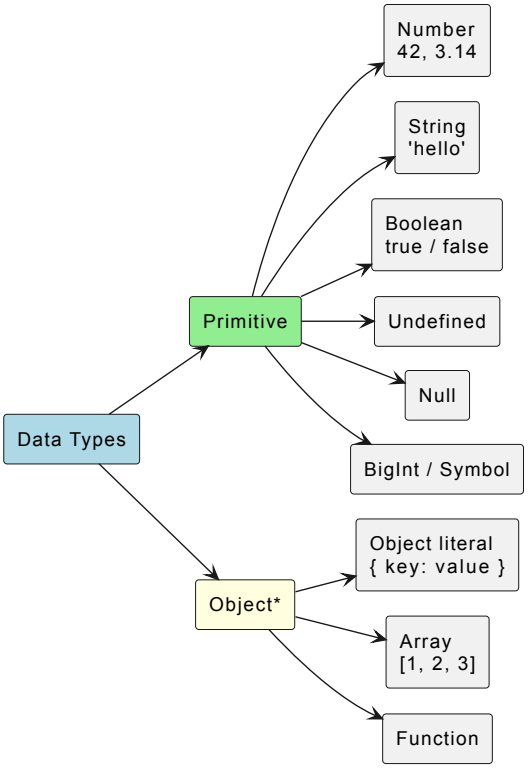
The Data Types of JavaScript

Every value in JavaScript is either a **primitive** or an **object**. Primitives are the basic single values: numbers, strings, booleans (which can only be `true` or `false`), `undefined`, `null`, `BigInt` and `Symbol`. Anything that is not a primitive is treated as an object: object literals, arrays, functions, and built-ins such as `Error`, `Date` or a regular expression.

Some of the most commonly used built-in objects include Object, Function, Boolean, Error, Number, Math, Date, String, and RegExp.

`undefined` vs `null`. The difference is subtle but specific. `undefined` means a variable exists but *no value has been given to it*. `null` is a special value that you assign deliberately to say "there is intentionally nothing here". If something is `null`, someone defined it that way.

Always declare with a keyword. Variables should always be defined using either the `var`, `const`, or `let` keywords.



Expression	Result
`typeof 42`	`"number"`
`typeof "hello"`	`"string"`
`typeof true`	`"boolean"`
`typeof undefined`	`"undefined"`
`typeof []`	`"object"`
`typeof {}`	`"object"`
`typeof null`	`"object"` (a famous historical quirk)

Checking a type at runtime. Because one variable can hold different kinds of values at different points in a program, `typeof` tells you what it holds *right now*: `typeof 42` is `"number"`, and `typeof []` is `"object"`.

> **Note for Review:** Pro-Tip: `typeof []` giving `"object"` shows that arrays are objects, so `typeof` cannot tell an array from a plain object. Also remember `typeof null` is `"object"` even though `null` is a primitive.

Output Methods

If you have written C you have used `printf`; in C++ you would use `cout`. These send the program's output to the *standard output*, which the operating system treats as just another file handle (a stream that can be redirected to the screen or to a file). JavaScript has the same idea, except that its output methods are built into the browser, and each one sends output to a *different place*.

Method	Description
`alert()`	Displays content within a browser-controlled pop-up/modal window.
`console.log()`	Displays content in the browser's JavaScript console.
`document.write()`	Outputs the content (as markup) directly to the HTML document.
`prompt()`	Displays a message and an input field within a modal window.
`confirm()`	Displays a question in a modal window with ok and cancel buttons.

- **`alert()`** is handy for automation-style messages: an event fires (say, a button is pressed) and you want to tell the user something, for example that form validation failed before data is sent to the server.
- **`console.log()`** is the developer's tool: output goes to the console, not to the visitor.
- **`document.write()`** sends the result of your script straight into the HTML document itself. It is mostly seen in small demos; modern code changes the page through the DOM instead (see Changing the DOM).
- **`prompt()`** collects input and **`confirm()`** collects an OK/Cancel decision, both in a modal window.

Operators

- **Arithmetic:** `+ - * / % **`, plus `+= -= *= /=` as shorthand for "update and reassign".
- **Comparison:** `===` and `!==` check value **and** type; `==` and `!=` coerce type first — best avoided.
- **Logical:** `&&` (and), `||` (or), `!` (not) — also handy for defaults, e.g. `name = name || "Guest"`.

Notice that equality is written with **three** equals signs (and "not equal" with `!==`), unlike the two used in C. The example multiplies a price by a quantity, $\text{total} = 250 \times 3 = 750$, and then asks the console two yes/no questions.

The first check, `total === 750`, is `true` because the value matches and both sides are numbers. The second combines two conditions with `&&`: the total exceeds 500 **and** the quantity is below 5, so the whole expression is `true`. Because `console.log` is used, the answers appear in the browser's JavaScript console.

```
let price = 250;
let qty = 3;
let total = price * qty;

console.log(total === 750);
// true
console.log(total > 500 && qty < 5);
// true
```

Why `===` beats `==`. Loose equality converts types before comparing, which produces surprises; strict equality does not.

Expression	Result	Reason
`5 === 5`	`true`	same value, same type
`"5" === 5`	`false`	string vs number
`"5" == 5`	`true`	`==` converts first (best avoided)

> **Note for Review:** Pro-Tip: exams love `==` vs `===`. Remember that `===` never converts types, so it behaves the way a C programmer expects `==` to behave.

Arrays

An **array** groups many values of any kind in one ordered container, accessed by a numeric *index* that starts at **0**. You can create it empty, create it pre-filled with comma-separated values inside square brackets, or store a value at a chosen position.

```
var name = [];                      // empty array
var name = [value, value, ..., value];   // pre-filled
name[index] = value;                // store element
```

The `length` of an array is always *the highest index in use plus one*. The example below shows this, including the case where we skip positions.

```
var school = ["stpauls", "dps", "vidyashram"];
var strength = [];              // strength.length is 0
strength[0] = "Ganga";          // strength.length is 1
strength[1] = "Yamuna";         // strength.length is 2
strength[4] = "Ravi";           // strength.length is 5
strength[4] = "Kaveri";         // strength.length is 5
```

(In the original slide these comments say `stooges.length`; it is the same array, `strength`.)

Statement	Array contents afterwards	`length`
`var strength = [];`	empty	0
`strength[0] = "Ganga";`	Ganga	1
`strength[1] = "Yamuna";`	Ganga, Yamuna	2
`strength[4] = "Ravi";`	Ganga, Yamuna, *(2 empty slots)*, Ravi	5
`strength[4] = "Kaveri";`	Ganga, Yamuna, *(2 empty slots)*, Kaveri	5 (overwrote index 4)

Writing into index 4 leaves positions 2 and 3 empty rather than failing, and writing to index 4 a second time *replaces* the value instead of growing the array.

Array Methods

An array can play the role of many data structures: a list, a queue, a stack, and more. A set of ready-made methods lets you add, remove and rearrange elements.

`push` adds an element at the **back** and **`pop`** removes the element at the back; this pair is the classic *stack* (last in, first out). **`unshift`** adds at the **front** and **`shift`** removes from the front; these do not exist in C or C++. Both `pop` and `shift` *return* the element that was removed. Finally, `sort` rearranges the array, here alphabetically (Blue before Red), by default comparing the elements as text.

```
var a = ["Red", "Blue"];
// Red, Blue
a.push("Green");
// Red, Blue, Green
a.unshift("Yellow");
// Yellow, Red, Blue, Green
a.pop();
// Yellow, Red, Blue
a.shift();
// Red, Blue
a.sort();
// Blue, Red
```

The methods available are: `concat`, `join`, `pop`, `push`, `reverse`, `shift`, `slice`, `sort`, `splice`, `toString`, `unshift`.

Method	What it does
`push` / `pop`	add / remove at the **back**
`unshift` / `shift`	add / remove at the **front**
`concat`	joins arrays into a new array
`join`	merges the elements into one string with a delimiter
`reverse`	reverses the order of the elements
`slice`	copies out a portion without changing the original
`splice`	adds or removes elements at any position
`sort`	sorts the elements
`toString`	converts the array to a string

Behaviour you want	Use
Stack (last in, first out)	`push` + `pop`
Queue (first in, first out)	`push` + `shift`

Strings

Text needs special handling, so JavaScript gives strings a rich set of methods. In the example, `indexOf(" ")` finds the position of the **first space**, and `substring(0, thatPosition)` extracts everything from position 0 up to (but not including) that spot: the first name.

In `"Albert Einstein"` the first space sits at index 6, so `substring(0, 6)` returns `"Albert"`. The `length` property counts every character including the space: $6 + 1 + 8 = 15$. Strings may be written with double quotes or single quotes, so `'IIT Jodhpur'` is just as valid. Splitting a long sentence this way is the first step in many downstream tasks.

- **Methods:** `charAt`, `charCodeAt`, `fromCharCode`, `indexOf`, `lastIndexOf`, `replace`, `split`, `substring`, `toLowerCase`, `toUpperCase`.
  - `charAt` returns a one-letter String (there is no `char` type).
- `length` is a **property** (not a method as in Java), so write `s.length`, never `s.length()`.
- Strings can be specified with `""` or `''`.
- Concatenation uses `+`:
  - `1 + 1` is `2`, but `"1" + 1` is `"11"`.

```
var s = "Albert Einstein";
var fName = s.substring(0,
            s.indexOf(" "));  // "Albert"
var len = s.length;           // 15
var s2 = 'IIT Jodhpur';
```

Method	Purpose
`charAt(i)`	the character at position *i*, as a one-letter string
`charCodeAt(i)`	the numeric character code at position *i*
`fromCharCode(n)`	builds a string from a character code
`indexOf(x)` / `lastIndexOf(x)`	first / last position of *x*
`replace(a, b)`	replaces text
`split(d)`	breaks the string into an array using delimiter *d*
`substring(i, j)`	the piece from *i* up to *j*
`toLowerCase()` / `toUpperCase()`	changes letter case

Escape Sequences and Conversions

Escape sequences behave as in Java: `\'` `\"` `\&` `\n` `\t` `\\`. Use `\n` for a new line, `\t` for a tab, and a backslash in front of a quote to print that quote inside a string of the same kind. (Of the list, `\&` is not a special code in JavaScript; it simply yields `&`.)

Mixing numbers and strings: using `+` with a string turns the number into text, so `"" + count` becomes `"10"`. In the other direction, `parseInt` pulls an integer out of the front of a string and `parseFloat` does the same for decimals; if no number can be found the result is `NaN` ("not a number").

```
var count = 10;
var s1 = "" + count;      // "10"
var s2 = count + " bananas, please!";
                          // "10 bananas, please!"
var n1 = parseInt("42 is the answer");   // 42
var n2 = parseFloat("hello");            // NaN
```

Accessing single letters:

```
var firstLetter = s[0];             // fails in IE
var firstLetter = s.charAt(0);      // does work in IE
var lastLetter = s.charAt(s.length - 1);
```

The last letter is at `s.length - 1` because counting starts at 0. (The `s[0]` warning is about the very old Internet Explorer; modern browsers accept it.)

Splitting and Joining

```
var s = "the quick brown fox";
var a = s.split(" ");
// ["the", "quick", "brown", "fox"]
a.reverse();
// ["fox", "brown", "quick", "the"]
s = a.join("!");
// "fox!brown!quick!the"
```

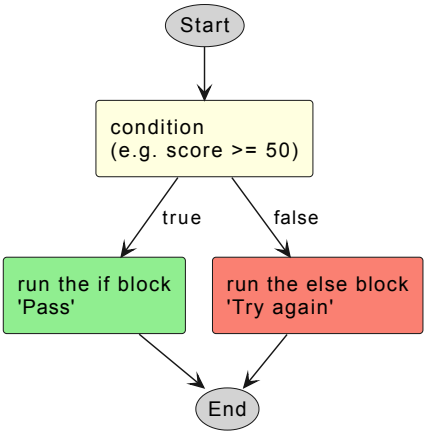
`split` breaks a string apart into an array using a delimiter; `join` merges an array back into a single string, placing a delimiter between the elements. Since `reverse` is an *array* method, the trick is: string → array → reverse → string.

> **Note for Review:** Pro-Tip: `split` makes an array from a string and `join` does the opposite; they are inverse partners. Remember that `"1" + 1` gives `"11"` while `"1" - 1` gives `0`, because only `+` doubles as string concatenation.

Control Flow

Programs become useful when they can make decisions: *if this happens, then do that*. In an `if` statement a **condition** is evaluated; if it is `true` the `if { }` block runs, otherwise the `else { }` block runs. Both paths then merge and the program continues.

In the diagram the condition is `score >= 50`. When it is true, the `if` block runs and reports "Pass"; when it is false, the `else` block runs and reports "Try again". Whichever path is taken, the program reaches the End. When you are comparing **one value against many options**, `switch(value) { case ... }` is a cleaner alternative to a long chain of `if / else`.



switch `checks the value against each` case `in turn and runs the block of the first match, very much as in C and C++. Each case normally ends with` break`; without it, execution "falls through" into the next case. (This example is illustrative.)

```
let score = 62;

if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Try again");
}
```

```
switch (day) {
  case "Sat":
  case "Sun":
    console.log("Weekend");
    break;
  default:
    console.log("Weekday");
}
```

Loops

When you need to repeat a set of statements, or repeat while a condition holds, loops automate the work. The ideas are the same as in other languages, so only the JavaScript flavours need attention:

- `for (...)` runs a fixed number of times — you control the counter yourself.
- `for...of` walks through each item of an array or string, in order.
- `while (...)` repeats for as long as a condition stays true — useful when you don't know the count in advance.

The first loop runs while `i` goes 1, 2, 3, printing "Round 1", "Round 2", "Round 3" (the backtick string with `${i}` inserts the variable's value into the text). The second loop uses `for...of`: the constant `fruit` takes each element of `fruits` in turn, so it runs exactly three times, and each time `toUpperCase()` converts the current fruit to capitals.

```
for (let i = 1; i <= 3; i++) {
  console.log(`Round ${i}`);
}

const fruits = ["apple", "mango", "kiwi"];
for (const fruit of fruits) {
  console.log(fruit.toUpperCase());
}
```

Pass	`fruit`	Output
1	`"apple"`	`APPLE`
2	`"mango"`	`MANGO`
3	`"kiwi"`	`KIWI`

A `while` loop (illustrative) counts down without a pre-set number of rounds:

```
let n = 3;
while (n > 0) {
  console.log(n);
  n--;
}
```

> **Note for Review:** Pro-Tip: choose the loop by the question you can answer. *"How many times?"* → `for`. *"For each item?"* → `for...of`. *"Until something changes?"* → `while`.

Functions

A **function** is a named, reusable block of code: you call it, optionally hand it some values, and it can hand a value back. These are user-defined functions, just as in other languages, with arguments going in and a return value coming out.

Every function has four parts. The **name** is its identifier, the handle by which you call it later. The **parameter** is a placeholder for the value the caller passes in; here it has a *default* (`name = "friend"`), which is used when the caller supplies nothing. The **body** is the block of code between the braces that runs each time the function is called. The **return value** is what is sent back to whatever called the function, via the `return` keyword. The backtick string `` `Hi, ${name}` `` slots the parameter's value into the text.

```
function greet(name = "friend") {
  return `Hi, ${name}`;
}

greet("Ivy")
// returns "Hi, Ivy"

greet()
// returns "Hi, friend"
```

Part	Meaning
name	the function's identifier — how you call it later
parameter (with default)	`name = "friend"` — a placeholder value the caller may override
function body	the block of code that runs each time the function is called
return value	sent back to whatever called the function, via the `return` keyword

Caller and callee. The function that makes a call is the *caller*; the function being called is the *callee*. If `main` invokes `greet("Ivy")` , then `main` is the caller. The callee runs its body, and when it finishes it returns a value to the caller, which carries on from where it left off, back in its own scope.

Declaring a Function: Two Styles

- **Function declaration** — `function greet(name) { return `Hi, ${name}`; }` — is **hoisted**, so it can be called before its definition appears in the file.
- **Function expression or arrow function:** `const greet = (name) => `Hi, ${name}`;` — **not hoisted**, and the style you'll see most in modern code. When the body is very short this is a tidy way to write it; the `=>` notation does not exist in C or C++.
- Parameters can carry defaults: `function greet(name = "friend") { ... }`
- Variables declared inside a function stay local to it — that's **function scope**, and it's what keeps different parts of a program from stepping on each other.

Hoisting means the browser notices function declarations before running any code, so calling `hello()` above its definition works. A `const` holding an arrow function only comes into existence when execution reaches that line, so calling `bye()` earlier fails.

```
hello();            // works
function hello() {
  console.log("hi");
}

bye();              // error
const bye = () =>
  console.log("bye");
```

Local means local. Suppose a function `grid` declares `let a = 20` , and another function also declares its own `let a` . These are two different variables: although the names match, each is limited to the body of the function that declared it. When a function returns, its scope is gone.

```
function grid() {
  let a = 20;
}

function main() {
  let a = 5;
  grid();
  console.log(a);   // 5: grid's a is a different a
}
```

Scope

Scope answers the question *"from where can this variable be seen, and for how long does it live?"* JavaScript has global scope, function (local) scope, and block scope. Four cases show how the keyword and the position of the declaration decide the outcome.

> A variable will be in **global scope** if declared outside of a function *and* outside of a block, regardless of the keyword used.

Case 1: `var` at the top level. `abc` is declared outside every function and block, so it is global. The function can read it (printing `fred`), and because it is the same variable, the function's assignment of `'sue'` is still there after the call returns.

```
var abc = 'fred';

function something() {
  console.log(abc);  // outputs: fred
  abc = 'sue';
}

something();
console.log(abc);    // outputs: sue
```

Case 2: `let` at the top level. The code is identical, with `let` replacing `var`, and the behaviour is the same. Outside any function and any block, the keyword does not change anything: the variable is global and the update to `'sue'` is visible everywhere.

```
let abc = 'fred';

function something() {
  console.log(abc);  // outputs: fred
  abc = 'sue';
}

something();
console.log(abc);    // outputs: sue
```

> A variable will be in **global scope** if declared outside of a function *and* uses the `var` keyword.

Case 3: `var` inside a loop. A `for` loop's braces are only a block, and `var` ignores blocks, so both `i` and `tmp` leak into the global scope. After the loop `i` holds 10 (it counted 0, 1, 2, ... up to 10, when the condition failed), and `tmp` still holds its last value, `"yes"`.

```
for (var i = 0; i < 10; i++) {
  var tmp = "yes";
  console.log(tmp);  // outputs: yes
}
console.log(i);      // outputs: 10
console.log(tmp);    // outputs: yes
```

> A variable declared within a `{}` block using `let` or `const` will have **block scope** and *only* be available within the block it is defined in.

Case 4: `let` and `const` inside a loop. Now `i` belongs to the loop and `tmp` to the loop body, so both vanish at the closing brace. Printing them afterwards raises an error ("is not defined").

```
for (let i = 0; i < 10; i++) {
  const tmp = "yes";
  console.log(tmp);  // outputs: yes
}
console.log(i);
// error: i is not defined
console.log(tmp);
// error: tmp is not defined
```

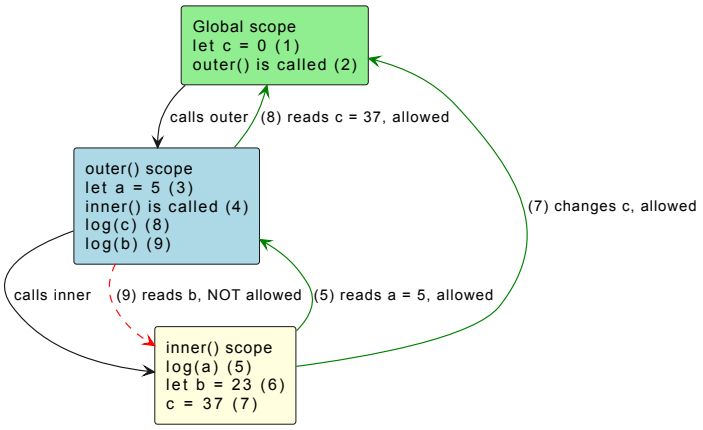
If you want the loop counter to survive, declare it *before* the loop, outside any block; it then belongs to the outer scope and would print 10 afterwards.

```
let i;
for (i = 0; i < 10; i++) { }
console.log(i);   // 10
```

Scope Inside Nested Functions

Scopes nest like boxes. The outermost box is the global block: anything declared there is global and accessible everywhere. Inside it, `outer()` has its own box where anything declared is accessible everywhere within that box. Inside that, `inner()` has the smallest box: anything declared in it is accessible only in that block. The rule is simple: **code can look outward, never inward.**

Follow the numbers in the code. `inner` reads `a` from the box around it (allowed, prints 5) and changes the global `c` to 37 (allowed). Back in `outer`, reading `c` shows 37 (allowed), but reading `b` fails, because `b` lives only inside `inner`.




```
let c = 0;                  // (1)
outer();                    // (2)

function outer() {
  function inner() {
    console.log(a);         // (5) outputs 5
    let b = 23;             // (6)
    c = 37;                 // (7) allowed
  }
  let a = 5;                // (3)
  inner();                  // (4)
  console.log(c);           // (8) outputs 37
  console.log(b);           // (9) not allowed
}
```

Step	What happens
1	global variable `c` is defined
2	global function `outer()` is called
3	local (outer) variable `a` is defined
4	local function `inner()` is called
5	local (outer) variable `a` is accessed from inside `inner` → outputs 5
6	local (inner) variable `b` is defined
7	global variable `c` is changed to 37
8	global variable `c` is accessed → outputs 37
9	undefined variable `b` is accessed → generates an error or outputs `undefined`

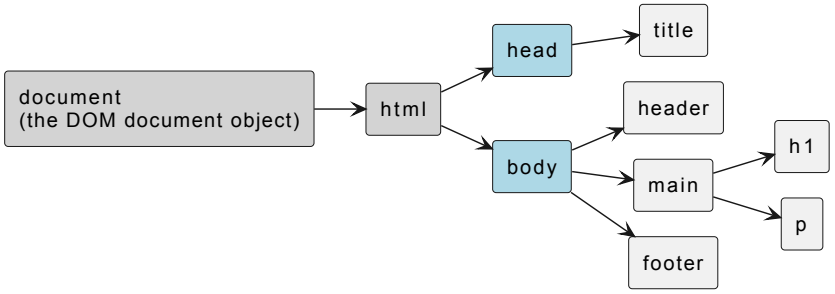
Step 2 can call `outer()` before the function is written in the file only because a function *declaration* is hoisted.

Where it is declared	Scope
Outside any function and block (any keyword)	Global
`var` inside a block such as a `for` loop	Global (if not inside a function)
`let` / `const` inside `{ }`	Block only
Anywhere inside a function	Local to that function

> **Note for Review:** Pro-Tip: this is a favourite "predict the output" topic. Remember that **`var` respects only function boundaries; `let` and `const` respect every pair of braces.** That is exactly why `var` "causes real bugs".

The Document Object Model

Now to a feature that makes JavaScript special on the web. You have already met the idea of a hierarchy in HTML and CSS; the **Document Object Model (DOM)** is that same hierarchy made available to JavaScript. The `document` is the topmost node, the root of the tree. Underneath it sits the `html` element; `head` and `body` are its two siblings; each has its own children, and the tree grows as the page nests deeper.



The DOM turns markup into a tree of node objects that JavaScript can read and change. Markup is just the tagging mechanism; the DOM converts every tag into an object that a script can find, read, and modify.

> DOM is a platform- and language-neutral interface that will allow programs and scripts to dynamically access and update the content, structure and style of documents. — W3C

The Toolbox at a Glance

- `document.querySelector("...")` — the modern way to find elements, using ordinary CSS-style selectors.
- `element.textContent` sets plain text; `element.innerHTML` parses and inserts HTML — powerful, but a real injection risk with untrusted input.
- `document.createElement("li")` plus `element.append(newEl)` builds a new node and inserts it into the page.
- `element.classList.add / remove / toggle("name")` — the standard way to change an element's CSS classes from script.

`innerHTML` inserts HTML *under the scope of a rooted subtree*, which means whatever markup is in the string becomes real elements. If that string came from a visitor, they can smuggle in their own markup or scripts. `textContent` treats the same string as inert text.

```
// userInput = '<img src=x onerror="alert(1)">'
el.textContent = userInput;
// shown as plain text: safe
el.innerHTML = userInput;
// parsed as HTML: injection risk
```

> **Note for Review:** Pro-Tip: this often appears on exams regarding *safe DOM updates*. Remember that `textContent` = text only, `innerHTML` = markup that gets *executed by the browser*; never feed `innerHTML` untrusted input.

Finding Elements in the DOM

Method	Description
`getElementById("id")`	Returns the single element node whose id attribute matches the passed id.
`getElementsByClassName("name")`	Returns a NodeList of elements whose class name matches the passed name.
`getElementsByTagName("name")`	Returns a NodeList of elements whose tag name matches the passed name.
`querySelector("selector")`	Returns the first element node that matches the passed CSS selector.
`querySelectorAll("selector")`	Returns a NodeList of elements that match the passed CSS selector.

Before you can change anything, you must **find** it: you first access (read) the element, then apply your changes. Different situations call for different selection methods.

> **Note for Review:** Pro-Tip: only `getElementById` and `querySelector` give you **one** element; the other three give you a **collection** that must be looped over. (Strictly, the `getElementsBy...` methods return a live `HTMLCollection` and `querySelectorAll` a static `NodeList`, but both behave as "a list of nodes".)

Selecting with the `getElement...` Methods

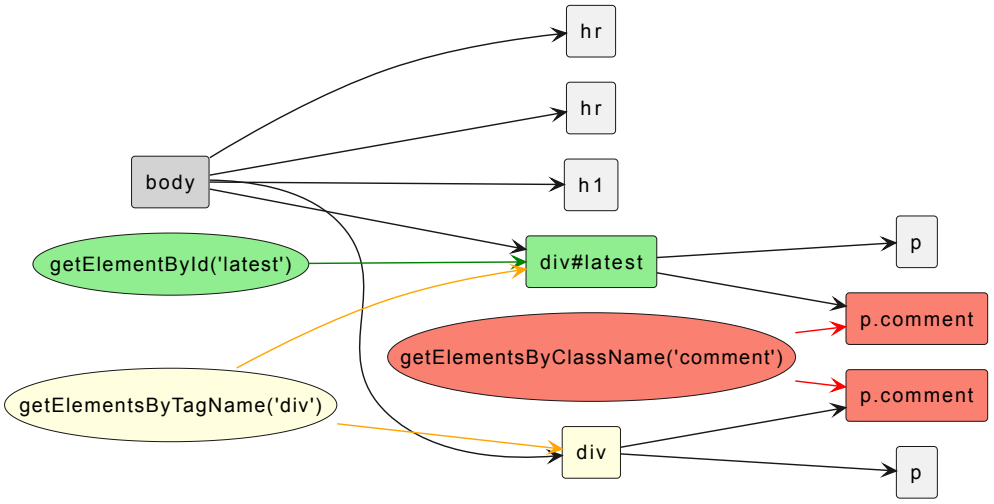
Take this partial page. These JavaScript lines may live in an external file; they can still reach into the HTML.

```
<body>
  <h1>Reviews</h1>
  <div id="latest">
    <p>By Ricardo on <time>2016-05-23</time></p>
    <p class="comment">Easy on the HDR buddy.</p>
  </div>
  <hr/>
  <div>
    <p>By Susan on <time>2016-11-18</time></p>
    <p class="comment">I love Central Park.</p>
  </div>
  <hr/>
</body>
```

```
let node  = document.getElementById("latest");
let list1 = document.getElementsByTagName("div");
let list2 = document.getElementsByClassName("comment");
```

- `getElementById("latest")` goes straight to the one `div` whose id is `latest`.
- `getElementsByClassName("comment")` collects every element carrying that class, wherever it sits — in either `div`.
- `getElementsByTagName("div")` makes a list of **all** the `div` elements; the tag name is the plain HTML tag name, without angle brackets.

The structure the script sees is a tree: `body` has `h1`, `div`, `hr`, `div`, `hr` as children, and each `div` has two `p` children.



Selecting with `querySelector` and `querySelectorAll`

These accept ordinary CSS selectors, so one method can express very precise searches.

```
<body>
  <nav>
    <ul>
      <li><a href="#">Canada</a></li>
      <li><a href="#">Germany</a></li>
      <li><a href="#">United States</a></li>
    </ul>
  </nav>
  <div id="main">
    Comments as of
    <time>November 15, 2012</time>
    <div>
      <p>By Ricardo on
         <time>September 15, 2012</time></p>
      <p>Easy on the HDR buddy.</p>
    </div>
    <div>
      <p>By Susan on
         <time>October 1, 2012</time></p>
      <p>I love Central Park.</p>
    </div>
  </div>
</body>
```

Call	Selects
`querySelectorAll("nav ul a:link")`	the three links (Canada, Germany, United States)
`querySelector("#main>time")`	the `time` that is a **direct child** of `#main` (November 15, 2012)
`querySelectorAll("#main div time")`	every `time` **inside a** `div` within `#main` (September 15 and October 1, 2012)

The difference between `>` and a plain space matters: `#main>time` means *direct child only*, while `#main div time` means *any descendant* `time` *that sits within a* `div`. That is why the November date is picked by the first and not by the second: it is a child of `#main`, not of an inner `div`.

Changing the DOM

Element Node Properties

Once you hold an element, its **properties** let you read and change it.

Property	Description
`classList`	A read-only list of CSS classes assigned to this element. This list has a variety of helper methods for manipulating this list.
`className`	The current value for the class attribute of this HTML element.
`id`	The current value for the id of this element.
`innerHTML`	Represents all the content (text and tags) of the element.
`style`	The style attribute of an element. This returns a CSSStyleDeclaration object that contains sub-properties that correspond to the various CSS properties.
`tagName`	The tag name for the element.

Changing an Element's Style

Retrieve the element first with `getElementById`, then set its CSS sub-properties through `style`. CSS names with a hyphen become camelCase (`background-color` → `backgroundColor`).

```
const node = document.getElementById("someId");
node.style.backgroundColor = "#FFFF00";
node.style.borderWidth = "3px";
```

To change *classes* instead (usually the better habit: styling stays in CSS), start from `var node = document.querySelector("main div");` on a `div` whose class is `box`:

#	Code	Effect	Resulting HTML
1	`node.className = "yellowish";`	Replaces the existing class specification, so the `<div>` no longer has the `box` class	`<div class="yellowish">`
2	`node.classList.remove("yellowish");` then `node.classList.add("box");`	Removes the specified class, then adds the `box` class	`<div class="">` then `<div class="box">`
3	`node.classList.add("yellowish");`	Adds a new class to the existing class specification	`<div class="box yellowish">`
4	`node.classList.toggle("hide");`	If it isn't in the class specification, then add it	`<div class="box yellowish hide">`
5	`node.classList.toggle("hide");`	If it is in the class specification, then remove it	`<div class="box yellowish">`

> **Note for Review:** Pro-Tip: `className = ...` **overwrites** every class, while `classList.add(...)` **adds one** and leaves the rest. `toggle` is a light switch: the first call turns it on, the second turns it off.

DOM Manipulation Methods

These methods add, remove or replace nodes, as siblings or as children, and let you restructure parent-child relationships.

Method	Description
`appendChild`	Adds a new child node to the end of the current node. `aParentNode.appendChild(newNode)`
`createAttribute`	Creates a new attribute node. `var newAttribute = document.createAttribute("name");`
`createElement`	Creates an HTML element node. `var newElement = document.createElement("tag");`
`createTextNode`	Creates a text node. `var newText = document.createTextNode("text content");`
`insertAdjacentElement`	Inserts a new child node at one of four positions relative to the current node.
`insertAdjacentText`	Inserts a new text node at one of four positions relative to the current node.
`insertBefore`	Inserts a new child node before a reference node in the current node. `aParentNode.insertBefore(newNode, referenceNode)`
`removeChild`	Removes a child from the current node. `aParentNode.removeChild(child)`
`replaceChild`	Replaces a child node with a different child. `aParentNode.replaceChild(newChild, oldChild)`

Visualizing a DOM Modification

Start with a small piece of HTML: a `div` with the id `first` holding a heading and one paragraph.

```
<div id="first">
  <h1>DOM Example</h1>
  <p>Existing element</p>
</div>
```

The goal is to add a new paragraph saying "this is dynamic". Building it takes four steps: **create** the pieces, **assemble** them, then **attach** the result to the page.

1. Create a new text node.
2. Create a new empty `<p>` element.
3. Add the text node to the new `<p>` element.
4. Add the `<p>` element to the `<div>`.

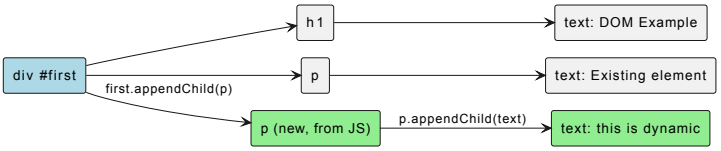
```
// 1: create a new text node
const text = document.createTextNode("this is dynamic");

// 2: create a new empty <p> element
const p = document.createElement("p");

// 3: add the text node to the new <p>
p.appendChild(text);

// 4: add the <p> to the <div>
const first = document.getElementById("first");
first.appendChild(p);
```

Until step 4 the new paragraph is only an object floating in memory; nothing changes on screen. The final `first.appendChild(p)` hooks it onto the `div` as its **last child**, so the `div` now has three children: the heading, the old paragraph and the new one, which appears after the existing element. The resulting HTML is shown below the tree.




```
<div id="first">
  <h1>DOM Example</h1>
  <p>Existing element</p>
  <p>this is dynamic</p>
</div>
```

> **Note for Review:** Pro-Tip: remember the **create** → **fill** → **attach** order. `createElement` and `createTextNode` only *make* nodes; nothing is visible until some `appendChild` (or `insertBefore`) attaches them to a node that is already in the page.

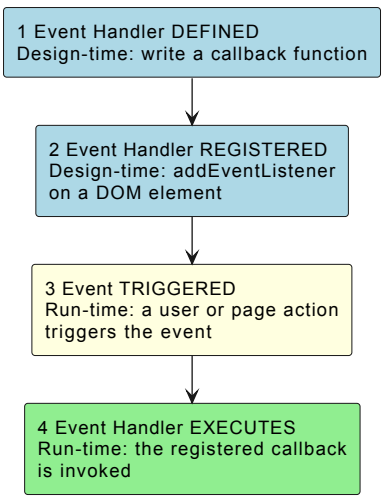
Events

JavaScript is **event-driven**, and this is its most important feature for client-side programming. A visitor is constantly interacting with the browser, giving input or looking at output, and every such interaction is an **event**: an event is something that happens in the browser — a click, a keypress, a page load, a timer firing. Right-clicking something, typing into a field, pressing a button: all events. Your job as the developer is to say *what should happen* when each one occurs.

The Four-Step Life of an Event

Registration works like signing up for a service: you subscribe **in advance**, and only when the event occurs are you notified. If you skip the sign-up step, nothing happens.

The first two steps happen at **design time**, when you write the code: you define a handler (a callback function saying what to do) and then register it for a given event on a given element with `addEventListener`. The last two happen at **run time**: a user or page action triggers the event, and the browser executes the registered handler, invoking your callback. Miss step 2 and step 4 never occurs.



In this example the handler `simpleHandler` is defined first (step 1) and simply raises an alert. Then the script finds the button by its id and **registers** the handler (step 2): `btn` is an object, `addEventListener` is its method, and the two arguments are *which event* (`"click"`) and *what to run* (`simpleHandler`). When the visitor clicks, the browser calls the handler and the message pops up. Notice that the function is passed **without parentheses**: you hand over the function itself, you do not run it yet.

```
<input type="submit" id="btn">
<script>
  // 1: event handler defined
  function simpleHandler() {
    alert("button was clicked");
  }

  const btn =
    document.querySelector("#btn");

  // 2: event handler registered
  btn.addEventListener("click",
                       simpleHandler);
</script>
```

Making a Page Respond

- An event is something that happens in the browser — a click, a keypress, a page load, a timer firing.
- `element.addEventListener("click", handler)` — attaches a response **without overwriting** any other listener already on that element.
- The event object passed into your handler describes what happened; `event.target` is the actual element that triggered it.
- `event.preventDefault()` — stops the browser's default behaviour, such as a form actually submitting.

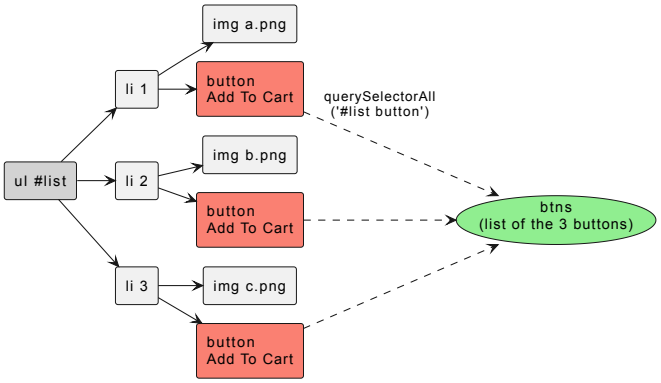
You use it in the same format for every element: *element, dot, method name*. Because listeners are added rather than replaced, you can stack several on the same element, whereas assigning to a property like `element.onclick = fn` would overwrite the previous one.

```
btn.addEventListener("click", (event) => {
  console.log(event.target);
  // the element that was clicked
});
```

> **Note for Review:** Pro-Tip: this often appears on exams regarding *registering handlers*. Remember `addEventListener("click", simpleHandler)` passes a **reference**; writing `simpleHandler()` with parentheses would run it immediately and register whatever it returns.

Events on Many Elements at Once

Imagine a shop page with three product cards, each with an "Add To Cart" button, built as a list (`<ul id="list">`) of `<li>` items. You want every button to react.




```
<ul id="list">
  <li>
    <img src="a.png" ... >
    <button>Add To Cart</button>
  </li>
  <li>
    <img src="b.png" ... >
    <button>Add To Cart</button>
  </li>
  <li>
    <img src="c.png" ... >
    <button>Add To Cart</button>
  </li>
</ul>
```

Remember that a node list (i.e., an array of nodes) doesn't support event listeners. Only individual node objects have the `addEventListener()` method defined. So the pattern is always: **select all → loop → attach to each**

```
// select all the buttons
const btns = document.querySelectorAll("#list button");

// this won't work and will generate error
btns.addEventListener("click", function () { ... });

// instead must loop through node list ...
for (let bt of btns) {
  // ...and assign event listener to each node
  bt.addEventListener("click", function () { ... });
}
```

Practical Scenarios

A Click Counter

This is a minimal but complete interactive widget that combines the three ingredients of almost every interactive page: a **DOM query**, an **event listener** and a **state update**, all working together.

The script first grabs the two elements it needs, the button (`#add`) and the paragraph that displays the number (`#count`), and creates a variable `count` starting at 0. It then registers a click listener on the button. Each click increases `count` by one and writes the new value into the paragraph with `textContent` . Because `count` is declared *outside* the handler, its value survives between clicks.

```
<button id="add">+1</button>
<p id="count">0</p>

<script>
  const button =
    document.querySelector("#add");
  const display =
    document.querySelector("#count");
  let count = 0;

  button.addEventListener("click", () => {
    count++;
    display.textContent = count;
  });
</script>
```

A Show/Hide Animation

The guiding rule here: **CSS defines the transition; JavaScript only flips a class. Style stays in CSS, behaviour stays in JS.**

In the stylesheet, a `.panel` starts collapsed (`max-height: 0` with `overflow: hidden`) and has a `transition` , so any change to its height is animated over 0.35 seconds. The `.open` class raises `max-height` to 300px. The script does nothing fancy: it finds the toggle button and the panel, and on every click `classList.toggle("open")` adds the class if it is absent and removes it if present, so the panel slides from hidden to visible and back. `max-height` is used instead of `display` because `display: none` cannot be animated.

```
/* styles.css */
.panel {
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.35s ease;
}
.panel.open { max-height: 300px; }
```

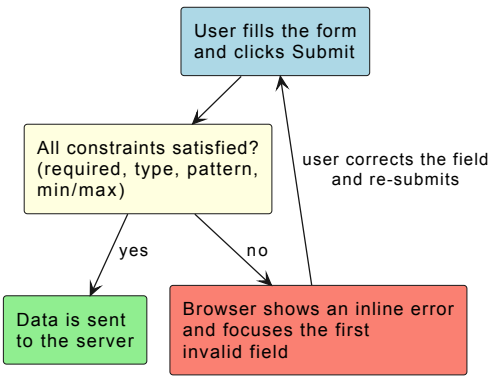
```
// script.js
const toggleBtn =
  document.querySelector("#toggle");
const panel =
  document.querySelector("#panel");

toggleBtn.addEventListener("click", () => {
  panel.classList.toggle("open");
});
```

Client-Side Form Validation

Before a form's data travels to the server, the browser can check that every **constraint** is satisfied: is the field required, is it the right *type* (an email, a phone number), does it match a *pattern*, is it within *min/max*? If everything passes, the data is sent; if not, the browser shows an inline error and focuses the first invalid field, and the user corrects it and submits again.

The loop in the flow is the heart of it: *fill, check, fix, resubmit*, until all constraints are satisfied. The check itself is done by the browser's built-in rules (required, type, pattern, min/max) before any data leaves the page. The important caveat is at the bottom of the slide: this is **a UX convenience, not security**, so always re-validate on the server too.



> **Note:** client-side validation is a UX convenience, not security -- always re-validate on the server too.

Validating a Signup Form

The browser's built-in **Constraint Validation API** checks `required` and `type` rules first; the handler below adds **one custom rule** on top. (Illustrative markup for the form being validated: an `<input type="email" required>`, a `<input type="password" name="password" required>`, and a submit button, inside `<form id="signup">`.)

The listener is attached to the form's `submit` event, and receives the `event` object. It reads the password from the field named `password`. If the length is below 8, it calls `event.preventDefault()`, which stops the form from actually submitting, shows an error message (`showError` is a helper you would write yourself), and `return`s to leave the handler. If the password is long enough nothing blocks the event, so the built-in checks have already passed and the form submits normally.

```
const form =
  document.querySelector("#signup");

form.addEventListener("submit",
  (event) => {
    const password = form.password.value;

    if (password.length < 8) {
      event.preventDefault();
      showError(
        "Password needs at least 8 characters");
      return;
    }
    // built-in checks already ran --
    // form submits normally
});
```

> **Note for Review:** Pro-Tip: exams like asking "what does `preventDefault()` do in a submit handler?" Remember that it **cancels the browser's default action** (sending the form), which is what lets your own validation say "no, not yet".

Asynchronous JavaScript

Synchronous vs Asynchronous

Back at the restaurant: a *synchronous* waiter takes your order, walks to the kitchen and **stands there** until the dish is ready, while every other table waits. An *asynchronous* waiter hands the order to the kitchen, goes back to serve the room, and comes over when the kitchen rings the bell.

- **Synchronous** code runs top to bottom, each line waiting for the one before it — a slow task blocks everything else on the page.
- **Asynchronous** code lets a long-running task, like a timer or a network request, run in the background while the rest of the page stays responsive.
- **A callback** is a function you hand over to run later, once the async task finishes.
- **Promises**, and the **async/await** syntax built on top of them, are the modern and more readable way to write asynchronous code.

With synchronous design there is a strict dependency between lines, so one slow task jeopardizes the performance of the entire pipeline. With the asynchronous style, you *offload* the work to the browser's Web APIs and carry on; when each task finishes, its result comes back and gets handled. The page stays responsive, many things can be in flight at once, and each follows its own course independently.

For reference (an illustrative sketch, not from the slides), the modern `async/await` style reads almost like synchronous code even though it never blocks the page:

```
async function loadStates(country) {
  const res = await fetch(`/api/states?c=${country}`);
  const data = await res.json();
  return data;
}
```

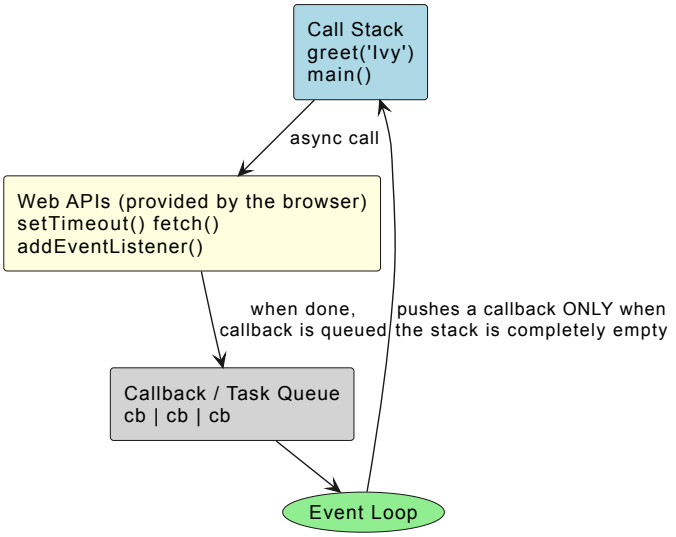
The Event Loop: How Async Code Runs

How does JavaScript do many things at once if only one thing runs at a time? Through four cooperating pieces:

- **Call Stack** — where running functions pile up: `main()` is at the bottom, and a call such as `greet("Ivy")` is stacked on top of it.
- **Web APIs** (provided by the browser) — `setTimeout()`, `fetch()` and `addEventListener()`. When your code makes an **async call**, it is handed over here and removed from the stack's concern.
- **Callback / Task Queue** — when an async task is done, its callback (`cb`) waits in line here.
- **Event Loop** — the watchman who moves callbacks from the queue onto the stack.

> **The rule:** the event loop only pushes a callback onto the call stack once the stack is completely empty.

Follow the arrows: the stack makes an async call to a Web API; the browser does the waiting; when finished, the callback is queued; and the event loop transfers it to the stack **only when nothing else is running**. This is why a long-running synchronous task delays even a timer set to zero seconds: the callback sits in the queue until the stack clears.

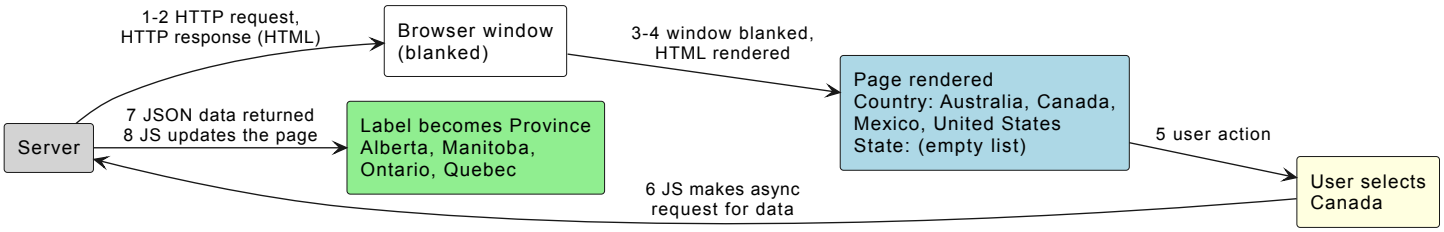



```
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");
// prints: A, C, B
```

> **Note for Review:** Pro-Tip: the "A, C, B" puzzle is a classic. Remember that `setTimeout(fn, 0)` does **not** mean "now"; it means "as soon as the stack is empty".

An Asynchronous Data Request in Action

Consider a form where choosing a **Country** fills in a **State/Province** list. Loading the entire page again for every choice would be slow; instead JavaScript fetches only the missing data.



1. **HTTP request** goes to the server.
2. **HTTP response** comes back, carrying the HTML.
3. After the browser receives the response, it blanks the browser window, and...
4. ...renders the just-received HTML in the browser window (the Country options, and an empty State list).
5. The user selects a country (say, Canada), then via JavaScript...
6. ...the browser makes an asynchronous request for data.
7. JSON data is returned.
8. Via JavaScript, the browser dynamically updates the document: it changes the label (State becomes Province) and populates the list from the JSON data (Alberta, Manitoba, Ontario, Quebec).

Re-loading the whole populated page for every choice would cost more latency and overhead. Here the visitor sees the page quickly (the first render doesn't wait for the extra data), and only the small piece that changed is fetched later. The result is a far more **responsive** page.

Putting It All Together

You can now describe a modern page end to end. JavaScript arrives as a file the browser downloads and runs. It stores values in variables (`let` / `const`), organises them in arrays and strings, makes decisions with `if` / `switch` , repeats with loops, and packages logic into functions with well-defined scope. Through the **DOM** it finds elements, reads them and rewrites them. Through **events** it reacts to the visitor. It can validate a form before data leaves, and through **asynchronous** requests it keeps the page responsive while data arrives. Throughout, remember the fine print: JavaScript can be switched off, stops at the first invalid line, and never replaces server-side checks.

Where we go next: the coming classes move to the **MVC architecture** and server-side programming.

Practice Questions

> Think before you look anything up. Sketch the code on paper, predict the behaviour, then test your answer in the browser console.

1. The buttons that went silent. A shop page runs `document.querySelectorAll("#list button")` when it loads and attaches a click listener to each of the 20 "Add To Cart" buttons in a loop. Later, a "Load more" button adds 20 more products to the list with `appendChild`, but clicking the new buttons does nothing.

- (a) Explain exactly why, using what `querySelectorAll` returns and *when* your listener-attaching code ran.
- (b) A teammate proposes one listener on the `ul` that uses `event.target` to find the clicked button. (Hint: an event on a child element is also reported to its ancestors.) Why would this keep working for products added later?
- (c) What check would you add inside that handler so clicks on the images or on blank space between cards are ignored?

2. Who survives the loop? Predict the output, and say which line (if any) stops the program and why.

- (a) What is printed, and does `"done"` ever appear? Relate your answer to how JavaScript treats an invalid line.
- (b) Change exactly one keyword so that `console.log(i)` fails the same way `bonus` does.
- (c) A colleague says, "Just use `var` everywhere so nothing is ever 'not defined'." Give two reasons this advice leads to bugs.

```
let total = 0;
for (var i = 1; i <= 3; i++) {
  let bonus = i * 10;
  total += bonus;
}
console.log(total);
console.log(i);
console.log(bonus);
console.log("done");
```

3. Zero seconds is not zero seconds. Consider:

- (a) In what order do A, B and C appear, and why does B wait even though the delay is 0?
- (b) While the big loop runs, what can the visitor do on the page? Explain using the call stack and the event loop rule.
- (c) If that heavy work were part of a real page, how would you restructure it so the page stays responsive?

```
console.log("A");
setTimeout(() => console.log("B"), 0);
for (let k = 0; k < 1e9; k++) { }
console.log("C");
```

4. The bypassed password rule. Your signup form checks `password.length < 8` in a `submit` handler and calls `event.preventDefault()`.

- (a) A tester disables JavaScript in the browser and submits a 3-character password; another person sends the HTTP request directly with a tool, skipping the page entirely. What happens in each case, and what does this say about where the rule must *also* live?
- (b) The page shows the error "Sorry, `<name>` is already taken", where `<name>` is whatever the visitor typed. Compare using `textContent` and `innerHTML` to display it if the visitor types `<b>x</b>`. Which is the safer default, and why?
- (c) Which checks has the browser already performed before your handler runs, and why can the handler safely assume them?

5. Design an accordion. You are building an FAQ page with 10 questions. Clicking a question should smoothly open its answer, and opening one should close any other that is open.

- (a) Which selection method collects all the question headings, and why can't you attach a listener to the collection directly?
- (b) Which part of the behaviour belongs in CSS and which in JavaScript? Name the `classList` operations you would use and where `toggle` is not enough.
- (c) The script file has a syntax error on line 40. What exactly does the visitor experience? How would you design the page so the questions are still readable?